



Projet Fondement de l'informatique

Ramdani Rayan
Picard Marius

10 décembre 2025

Table des matières

- 1. Introduction**
 - 1.1 Objectif du projet**
 - 1.2 Problématique du stockage**
- 2. Structure du projet**
- 3. Calcul d'histogramme couleur**
 - 3.1 Manipulation de listes**
 - 3.2 Construction, initialisation, destruction de l'histogramme**
 - 3.3 Interrogation de l'histogramme**
- 4. Implémentation de ppmhist**
- 5. Quantification par popularité**
- 6. Question optionnelle : Regroupement de couleurs**
- 7. Conclusion**

Il faut stocker la fréquence de chaque couleur (R, G, B) présente dans une image. Pour se faire, la solution envisagée est un tableau `histo[256][256]` où `histo[R][G]` pointe vers une liste chaînée triée des valeurs de B avec leurs fréquences.

`histo[R][G] → [B=2, freq=f1] → [B=6, freq=f2] → [B=8, freq=f3] → NULL`

3.1. Choix structure de donnée

La structure de données utilisés est la suivante, une liste chaînée de cellules :

```
typedef struct cell *cell;
struct cell {
    unsigned char B;        // Valeur de la composante Bleue (0-255)
    int freq;               // Fréquence d'apparition
    struct cell *next;      // Pointeur vers cellule suivante
};
```

Le choix de l'utilisation de cellules permet de ne pas réserver de mémoire en trop pour des couleurs qui ne seraient pas dans l'image.

Nous avons implémenté les fonctions suivantes :

1. `cell create_cell(int B, cell next)`

Alloue dynamiquement une cellule et initialise : -B : valeur fournie -next : pointeur fourni -freq : 1

2. `cell insert_cell(cell head, int B)`

Insère B dans la liste triée par ordre croissant : - Si B existe déjà : incrémente freq - Sinon : créé et insère une nouvelle cellule en faisant attention que les cellules restent triés.

La complexité est $O(n)$ où n est le nombre de cellules déjà présentes pour R et G.

3. `cell delete_list(cell list)`

Libère toute la mémoire d'une liste en utilisant un `while list!=NULL` afin de ne pas faire de test inutile. Sa complexité est aussi $O(n)$ où n est le nombre de cellules déjà présentes pour R et G.

3.2. Construction, initialisation, destruction de l'histogramme

Nous avons implémenté le type `histo` qui est un tableau de pointeur sur cellule dans `histogramme.h`.

```
typedef cell **histo; // Tableau 2D de pointeurs sur listes
```

Nous avons implémenté les fonctions suivantes :

1. `histo create_histo(void)`

Alloue dynamiquement un tableau 256×256 initialisé à NULL à l'aide d'une double boucle for. Sa complexité est donc $O(256^2) = O(1)$

2. `void init_histo(histo h, image img)`

Cette Fonction initialise l'histogramme à partir d'une image. Elle effectue les opérations suivantes :

Pour chaque pixel (i, j) de l'image :

1. Récupérer (R, G, B) du pixel
2. Si `h[R][G] == NULL` :
→ Créer liste avec B (fréquence = 1)
3. Sinon :
→ Insérer B dans `h[R][G]` (incréménte si existe)

```
void init_histo(histo histo, image img) {
    int i, j, R, G, B;
    int* pixel_rgb;
    int hauteur = image_give_hauteur(img);
    int largeur = image_give_largeur(img);

    for (i = 0; i < hauteur; i++) {
        for (j = 0; j < largeur; j++) {
            pixel_rgb = image_get_pixel(img, i, j);
            R = pixel_rgb[0];
            G = pixel_rgb[1];
            B = pixel_rgb[2];

            if (histo[R][G] == NULL) {
                histo[R][G] = create_cell(B, NULL);
            } else {
                histo[R][G] = insert_cell(histo[R][G], B);
            }
        }
    }
}
```

Sa complexité est $O(n \times m)$ où n est le nombre de pixels et m est la longueur moyenne des listes. Dans le pire des cas : $m = 256$, donc $O(256n)$. Mais en réalité les listes de cellules sont plus petites.

3. void delete_histo(histo h)

Libère toute la mémoire :

```
void delete_histo(histo h) {
    int i, j;
    for (i = 0; i < 256; i++) {
        for (j = 0; j < 256; j++) {
            delete_list(h[i][j]);
        }
        free(h[i]);
    }
    free(h);
}
```

Sa complexité est $O(256^2 + c)$ où c est le nombre total de couleurs.

2.3. interrogation de l'histogramme

Ces fonctions vont permettre de récupérer les informations stockées dans l'histogramme comme la fréquence de chaque triplet RGB. ou tout simplement quels triplets RGB sont présents. Ces fonctions seront également utiles pour récupérer les triplet RGB pour les autres fichiers .c qui n'ont pas accès à la structure de cellule.

```
int give_freq_histo(histo h, int R, int G, int B)
```

Retourne la fréquence d'une couleur (R, G, B) :

```
int give_freq_histo(histo h, int R, int G, int B) {
    cell curr;

    if (h == NULL)
        return 0;

    curr = h[R][G];
    while (curr != NULL) {
        if (curr->B == B)
            return curr->freq;
        curr = curr->next;
    }
    return 0;
}
```

Sa complexité est la taille de $h[R][G]$ donc $O(m)$.

Afin de parcourir l'histogramme nous avons mis en place un itérateur qui enregistre les valeurs de R et G ainsi que la cellule courante ou l'on se trouve. Son intérêt est de parcourir automatiquement toutes les couleurs présentes dans l'histogramme sans que le code utilisateur ait à gérer manuellement les trois niveaux de boucles imbriquées (R, G, liste de B) et les entrées vides. Il est essentiel dans la fonction quantification() que nous verrons plus tard afin d'extraire les K couleurs les plus fréquentes en parcourant séquentiellement l'histogramme de manière simple et fiable, sans erreurs d'indexation.

```
typedef struct histo_iter *histo_iter;
struct histo_iter {
    int R, G;           // Position courante dans le tableau
    cell current;       // Cellule courante dans la liste
};
```

1. histo_iter create_histo_iter(histo h)

Cette fonction initialise l'itérateur en cherchant la première couleur à l'aide de while.

```
while (h[i][j] == NULL && i < N) {
    j++;
```

```
if (j == N) {
    j = 0;
    i++;
```

2. void start_histo_iter(histo_iter iter, histo h)

Repositionne l'itérateur au début (même code que `create_histo_iter` sans allocation).

3. boolean next_histo_iter(histo_iter iter, histo h)

Avance l'itérateur à la couleur suivante, elle regarde d'abord si elle peut simplement accéder à la cellule suivante. Sinon elle cherche à l'aide d'un double while sur R et G la prochaine cellule non vide.

```
while (i < N) {
    while (j < N) {
        if (h[i][j] != NULL) {
            iter->R = i;
            iter->G = j;
            iter->current = h[i][j];
            return true;
        }
        j++;
    }
    j = 0;
    i++;
}
```

Sa complexité n'est pas vraiment importante ici, elle parcourt l'histo de manière à ne pas manquer de couleurs.

4. void give_color_histo_iter(histo_iter iter, int* couleur)

Récupère la couleur courante de l'itérateur et l'enregistre dans couleur

5. void delete_histo_iter(histo_iter iter)

Libère l'itérateur.

4. Implémentation du ppmhist

L'intérêt de cette partie est de créer un programme parcourant l'histogramme à l'aide de `histo_iter` afin d'afficher le triplet RGB la luminosité associée et le nombre d'occurrence du triplet. Nous avons créé la fonction suivante ainsi qu'un menu directement dans le main afin d'effectuer cette tâche.

1. void histo_display(char* chemin_image)

Cette fonction affiche l'histogramme dans le terminal. Elle prend en argument le nom de l'image que l'on traite. A l'aide du module du professeur elle crée une image la charge puis construit l'histogramme puis elle le parcourt à l'aide de l'itérateur et l'affiche dans le terminal.

```
void histo_display(char* chemin_image) {
    // Construction du chemin complet
    // Chargement de l'image
    // Construction de l'histogramme
    // Affichage
    printf("INFOS HISTOGRAMME :\nR, G, B, Luminosité, Fréquence\n");
    do {
        give_color_histo_iter(iter, couleur);
        lum = (couleur[0] + couleur[1] + couleur[2]) / 3;
        freq = give_freq_histo(h, couleur[0], couleur[1], couleur[2]);
        printf("%d, %d, %d, %d, %d\n",
            couleur[0], couleur[1], couleur[2], lum, freq);
    } while (next_histo_iter(iter, h));
    // Libération mémoire
}
```

2. void main()

Notre menu est interactif via le terminal il fonctionne à l'aide d'un switch case. Chacune des propositions explique ce qu'il faut faire. L'utilisateur n'a qu'à se laisser guider. Le menu est conçu de manière à pouvoir rajouter n'importe quelle option de manière simple sans avoir à repenser sa structure. Il utilise la fonction menu_display suivante :

```
void menu_display(char * menu[]){
    int i = 1;
    printf("%s", menu[0]);
    while(menu[i] != NULL){
        printf("%d. %s\n", i, menu[i]);
        i++;
    }
}
```

Ainsi, si l'on ajoute une ligne dans le tableau de chaînes de caractères ainsi que le case associé, il n'y a rien à faire de plus.

5. Quantificateur par popularité

L'objectif de cette partie est de réduire une image à K couleurs. On le fait en sélectionnant les K couleurs les plus fréquentes puis en remplaçant chaque pixel par la couleur la plus proche dans cette palette. Pour trouver la couleur la plus proche nous avons implémenter deux méthodes ainsi qu'une optimisation qui seront détaillées dans la section sur mapping.

5.1. Quantification

```
int* quantification(histo h, int* tab, int K)
```

Voici l'explication clair de comment fonctionne notre algorithme :

1. Initialiser un tableau des K premières couleurs triées par fréquence
→ Utiliser un tableau `freqs[K]` pour stocker les fréquences
2. Pour les K premières couleurs de l'histogramme :
 - a. Insérer dans `tab` en maintenant l'ordre décroissant des fréquences à l'aide du tri par insertion
3. Pour chaque couleur restante (R, G, B, f) :
 - a. Si $f \leq \text{freqs}[K-1]$: ignorer
 - b. Sinon tri par insertion :
 - Trouver position d'insertion i (tri par insertion)
 - Décaler `tab[i..K-2] → tab[i+1..K-1]`
 - Insérer (R, G, B) à la position i
4. Retourner `tab` contenant les K couleurs

Notre code du tri par insertion :

```
// Tri par insertion
while (indice > 0 && freqs[indice-1] < freq_temp) {
    freqs[indice] = freqs[indice-1];
    tab[indice*3] = tab[(indice-1)*3];
    tab[indice*3+1] = tab[(indice-1)*3+1];
    tab[indice*3+2] = tab[(indice-1)*3+2];
    indice--;
}
tab[indice*3] = couleur_temp[0];
tab[indice*3+1] = couleur_temp[1];
tab[indice*3+2] = couleur_temp[2];
count++;
```

La complexité de cet algorithme est celle du tri pas insertion que nous avons utilisé ici car il est optimal si $K < 1000$ sinon un tri par tas aurait pu être intéressant pour $K > 1000$.

La complexité est donc pour les K premiers éléments en $O(K^2)$. Puis pour le parcours des couleurs restantes en $O(c * K)$. La complexité finale est donc $O(cK + K^2)$ or $c \gg K$ donc $O(cK)$ avec c le nombre de couleur restante. Cette complexité est élevée mais faible par rapport à mapping.

5.2. Mapping

Dans cette partie l'objectif était de trouver dans les K couleurs les plus présentes. La couleur la plus proche du triplet du pixel. Pour déterminer cette couleur nous avons d'abord utilisé un principe simple, la distance de Manhattan qui revient à simplement calculer les normes euclidiennes dans l'espace 3D RGB entre notre point et les K points des couleurs les plus présentes. et de garder la plus faible.

Nous avons ensuite fait des recherches et avons appris que l'œil humain ne percevait pas les couleurs de manière proportionnelle et que donc ce modèle n'était pas le plus adapté. Nous avons donc essayé de convertir le triplet RGB en valeur CIELAB afin de comparer la norme entre les valeurs de notre couleur et celle des K couleurs dans un espace plus adapté. Cependant, la calcul demande beaucoup d'opérations donc bien que le rendu soit un peu mieux, l'algorithme est bien plus lent mais nous l'avons tout de même laissé.

1. mapping1 Manhattan

La fonction parcourt chaque pixel, on cherche, parmi les K couleurs les plus présentes, celle qui est la plus proche du triplet RGB du pixel. Chaque couleur est représentée dans l'espace RGB par un triplet (R,G,B). La proximité entre deux couleurs est évaluée à l'aide de la distance de Manhattan dans l'espace RGB, définie par :

$$d=|R_1 - R_2| + |G_1 - G_2| + |B_1 - B_2|$$

Pour chaque pixel, cette distance est calculée avec chacune des K couleurs de la palette, puis la couleur associée à la distance minimale est sélectionnée.

Comme la fonction parcourt chaque pixel de l'image et que pour chaque pixel elle parcourt chacune de K couleurs sa complexité est : $O(n \times K)$ Pour une image de 512×512 pixels $n = 262\,144$ ce qui est très élevé. Pour un $K = 100$ la fonction met environ 1 sec pour générer l'image.

Pour remédier à ce problème, nous aurions pu ne pas utiliser l'algorithme du plus proche voisin, mais plutôt une structure de données comme un octree (ou un k-d tree) permettant de réduire fortement le nombre de comparaisons en organisant les couleurs dans l'espace RGB, ce qui accélère la recherche de la couleur la plus proche.

2. mapping2 CIELAB

Mapping 2 fonctionne de la même manière que Mapping 1, il y a juste en plus les fonctions permettant la conversion de RGB vers CIELAB qui suivent les étapes suivantes (cf https://fr.wikipedia.org/wiki/%C3%89cart_de_couleur, <https://fr.wikipedia.org/wiki/SRGB> et https://fr.wikipedia.org/wiki/L*a*b*_CIE_1976) :

- Normalisation $R_n = R / 255$
- Gamma inverse
- Transformation RGB $\rightarrow$ XYZ
- Normalisation par blanc D65
- Fonction $f(t)$
- Lab final

La complexité de cette fonction est bien plus importante que celle de mapping 1 à cause du nombre d'opérations pour transformer en CIELAB. Son utilisation est donc à modérer car l'érésultat n'est pas beaucoup mieux.

La complexité de mapping2 est $O(n \times K \times C_lab)$ où n est le nombre de pixels, K le nombre de couleurs de la palette, et $C_lab \approx 60$ opérations représente le coût de la conversion RGB $\rightarrow$ CIELAB et du calcul de distance euclidienne, ce qui donne environ 1,6 milliard d'opérations pour une image 512×512 avec $K=100$. Ce qui prend environ 6 à 10 secondes et bien plus si K augmentes. Donc assez long. ON recommande donc d'utiliser mapping 1 pour un $K > 500$.

3. void Programme_quantification 1 et 2

Les deux fonctions mapping et la fonction quantification fonctionnent à l'aide de cette fonction. On appelle cette fonction dans le main afin de faire fonctionner le tout. Elle s'occupe notamment de créer l'image à partir du chemin donné par l'utilisateur.

6. Question Optionnelle : regroupement de couleurs

Pour implémenter cette optimisation nous avons créé la fonction suivante:

```
void rgb_bonus(int R, int G, int B, int* qR, int* qG, int* qB) {  
    *qR = (R / 2) * 2;  
    *qG = G;  
    *qB = (B / 4) * 4;  
}
```

Cette fonction permet de réduire le nombre de couleurs ressemblantes et d'avoir un panel K plus varié. En effet par exemple les couleurs le même R et G qui ont un B = 0, 1, 2 ou 3 seront considérées identiques.

Nous avons rajouté ensuite dans quantification l'appel à rgb_bonus afin de modifier la valeur des variables que nous utilisons puis nous remplissons le tableau des K couleurs normalement.

7. Conclusion

Ce projet a permis de consolider nos connaissances en structures de données dynamiques, d'approfondir les algorithmes de traitement d'image, découvrir les espaces colorimétriques perceptuels et améliorer notre rigueur en programmation C (gestion mémoire, débogage)

La partie la plus intéressante a été la conversion CIELAB et la constatation de son impact visuel peu significatif pour un temps d'attente excessivement long. La comparaison quantitative entre les deux méthodes a démontré l'importance de choisir des méthodes adaptées au problème.